

Spotify Content-Based Recommendation System

Project idea and machine-learning pipeline

Core question: If a user likes one song, can we recommend other songs with similar musical characteristics?

1. Project idea

The project would build a **content-based music recommender** using the 2023 Spotify dataset linked in the project repository. Instead of using other users' listening histories, the system would recommend songs based on measurable characteristics of the songs themselves.

Each song is represented by an audio-feature vector. A simple version could use:

Feature	What it captures
Danceability	How suitable the track is for dancing
Energy	Perceived intensity/activity
Valence	Musical positivity or mood
Acousticness	Amount / likelihood of acoustic sound
Instrumentalness	Degree of instrumental rather than vocal content
Liveness	Presence of live-performance characteristics
Speechiness	Amount of spoken-word content
BPM	Tempo in beats per minute

For song i , the representation can be written schematically as $\mathbf{x}_i = (\text{danceability}, \text{energy}, \text{valence}, \text{acousticness}, \text{BPM}, \dots)$. The recommendation task is to find the songs whose feature vectors are closest to the selected song.

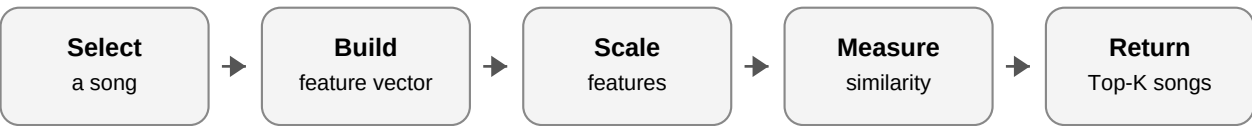
Why this is interesting

The output is tangible: a user chooses a song and receives a ranked list of similar songs. It uses machine learning for a clear purpose - similarity search in a multidimensional feature space - rather than adding a model merely for complexity.

What kind of recommender is this?

This is a **content-based recommender**. It is **not collaborative filtering**, because the linked dataset does not provide a user-by-song matrix of ratings, likes, or listening histories.

2. Machine-learning pipeline



Step	What we do	Why it matters
1. Select features	Choose the audio characteristics that define musical similarity.	The definition of similarity depends on the variables included.
2. Clean data	Handle missing values, duplicates, and non-numeric formatting.	Similarity methods require consistent numeric inputs.
3. Standardize	Convert features to comparable scales, for example $z = (x - \text{mean}) / \text{SD}$.	BPM may be around 100-180 while audio percentages are on a 0-100 scale; unscaled features can dominate distance.
4. Build song vectors	Represent every song as a point in p-dimensional feature space.	This gives the recommender a mathematical representation of each track.
5. Compute similarity	Use cosine similarity or a distance metric used by K-nearest neighbors.	The method quantifies how close two songs are in their audio profiles.
6. Rank neighbors	For a selected song, rank all other songs from most to least similar.	The nearest songs become the recommendation candidates.
7. Return Top-K	Display, for example, the 5 or 10 most similar tracks.	This converts the model output into an actual recommendation system.

Core ML methods

K-Nearest Neighbors (KNN). For a selected song, KNN finds the K observations closest to it in feature space. With Euclidean distance, songs are closer when their standardized audio features differ by smaller amounts across the selected dimensions.

Cosine similarity. This measures the angle between two feature vectors: $\text{sim}(\mathbf{x}, \mathbf{y}) = (\mathbf{x} \cdot \mathbf{y}) / (||\mathbf{x}|| ||\mathbf{y}||)$. A larger value indicates a more similar feature profile.

A useful implementation strategy is to test both methods and inspect whether the recommendation lists are stable and musically plausible. The final system can use whichever similarity definition produces more coherent neighbors.

3. What the final project would produce

Deliverable	Example
Exploratory analysis	Distributions and relationships among danceability, energy, valence, BPM, etc.
Similarity model	Standardized feature matrix + KNN/cosine similarity
Recommendation function	Input one track name; return its Top-5 or Top-10 nearest songs
Interpretation	Explain why recommended songs are close using their feature differences
Optional interface	A small R Shiny, Streamlit, or notebook interface for selecting a song

How could we assess the recommender?

Because this dataset does not contain individual users' ratings or listening histories, we cannot honestly claim that we can measure whether a person *likes* the recommendations using standard collaborative-filtering metrics. Instead, evaluation should focus on the structure of the recommender: inspect nearest-neighbor consistency, compare different feature sets and similarity metrics, and perform qualitative checks on whether recommended tracks genuinely have similar audio profiles.

Useful extensions - only if the group wants more depth

PCA: reduce correlated audio features to a smaller set of components before calculating similarity.

Clustering: group songs into broader musical profiles and compare cluster-based recommendations with direct nearest-neighbor recommendations. **Weighted similarity:** allow certain features to matter more; for example, a user could emphasize mood (valence) or intensity (energy).

Concise group pitch

"My idea is to build a content-based Spotify recommendation system. The 2023 dataset contains audio characteristics such as danceability, energy, valence, acousticness, instrumentalness, liveness, speechiness and BPM. We can represent each song by these features, standardize them, and use K-nearest neighbors or cosine similarity to find the songs with the most similar audio profiles. The final product would let someone select a song and return the Top-K most similar tracks."

Sources

Project repository: <https://github.com/megmillet/Spotify-Project>

Linked 2023 dataset: Kaggle - Most Streamed Spotify Songs 2023. The repository also links an example Spotify music recommendation-system notebook for reference.

Important scope note: the 2023 file contains already-popular songs, so the recommender's candidate pool is limited to songs represented in that dataset. A production Spotify recommender would require a much broader catalog and, for personalized recommendations, user-behavior data.